Ficha Técnica: Agente Autónomo de Triaje de Soporte (TypeScript + n8n)

Oct 9, 2026 · @Wenceslao José Villegas

Descripción general

Servicio backend headless que actúa como empleado digital de soporte: recibe solicitudes por webhook o API REST, decide qué herramientas ejecutar y resuelve el caso. Las acciones riesgosas requieren aprobación humana.

El trabajo se reparte en dos capas con responsabilidades distintas:

- **n8n (borde del sistema):** recibe eventos de Slack, email o formularios, los normaliza, envía notificaciones y gestiona las aprobaciones humanas.
- **Agente en TypeScript (núcleo):** contiene el loop agéntico, el registro de tools, la auditoría y las evaluaciones.

Esta división muestra dos habilidades en un solo proyecto: programación backend con control total del agente, e integración rápida de flujos de automatización.

Stack tecnológico

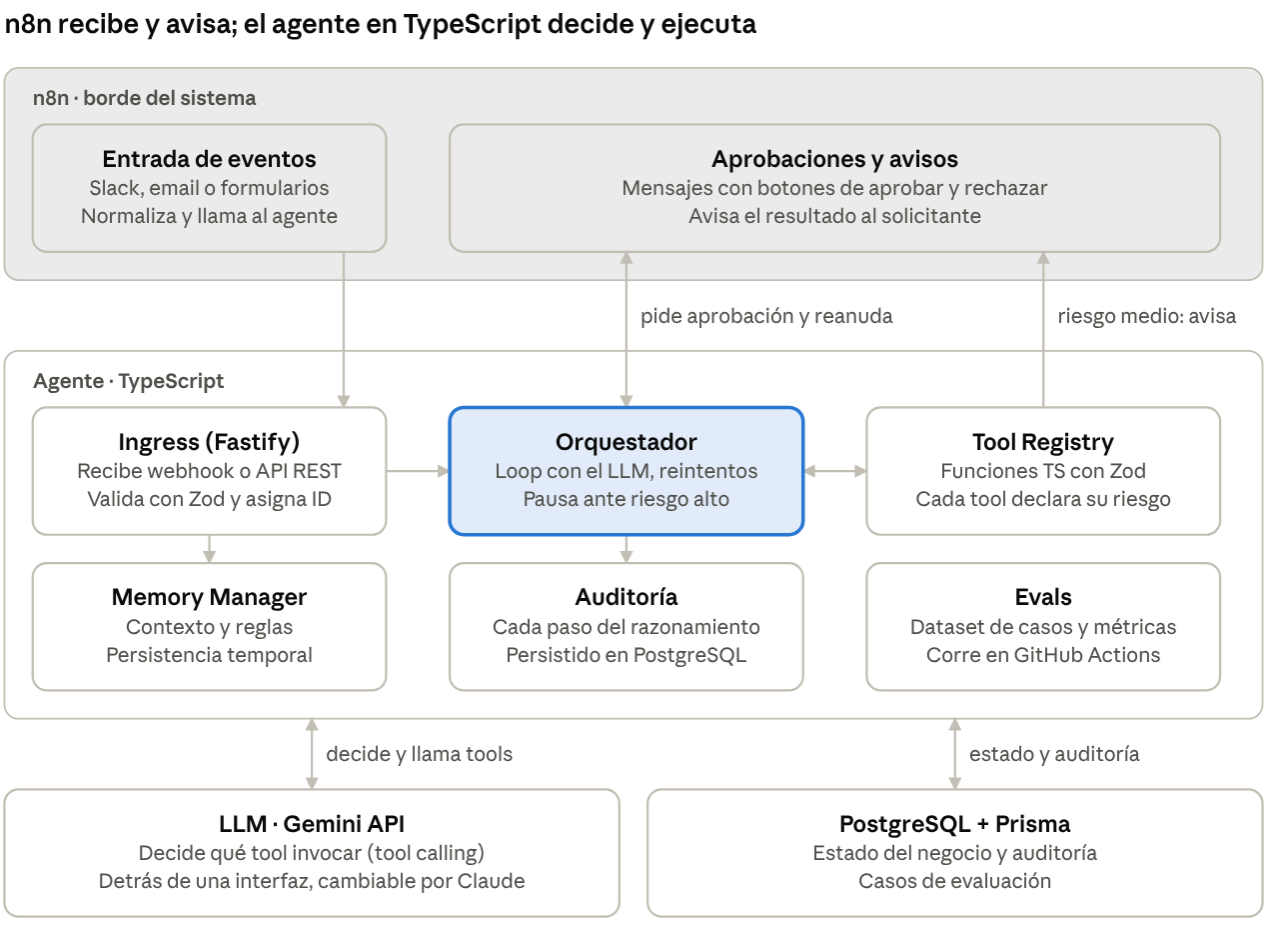
TypeScript estricto para los contratos de datos del agente, y n8n para todo lo que sea integración y flujos de aprobación.

Capa	Tecnología	Para qué se usa
Lenguaje	TypeScript (Node.js)	Tipado estricto en tools, mensajes y contratos de datos
Servidor HTTP	Fastify	Endpoint de ingreso, healthcheck y callbacks de aprobación
Motor de IA	Gemini API (Claude como alternativa)	Tool calling nativo; herramientas expuestas bajo el estándar MCP
Validación	Zod	Esquema de cada tool y validación de los argumentos que genera el modelo
Datos	PostgreSQL + Prisma	Estado del negocio, logs de auditoría y casos de evaluación

Capa	Tecnología	Para qué se usa
Automatización	n8n	Webhooks de entrada, notificaciones y aprobaciones humanas
Testing	Vitest	Tests unitarios de tools y runner de evals
Infraestructura	Docker + docker-compose	Agente, PostgreSQL y n8n levantados con un solo comando
CI/CD	GitHub Actions	Lint, tests, evals y build de la imagen

El agente no depende de un modelo concreto: el orquestador habla con una interfaz propia (`LLMProvider`) y cada proveedor tiene su adaptador. Se arranca con Gemini usando una API key de Google AI Studio, que es independiente de la suscripción a la app de Gemini, y se puede pasar a Claude con otro adaptador.

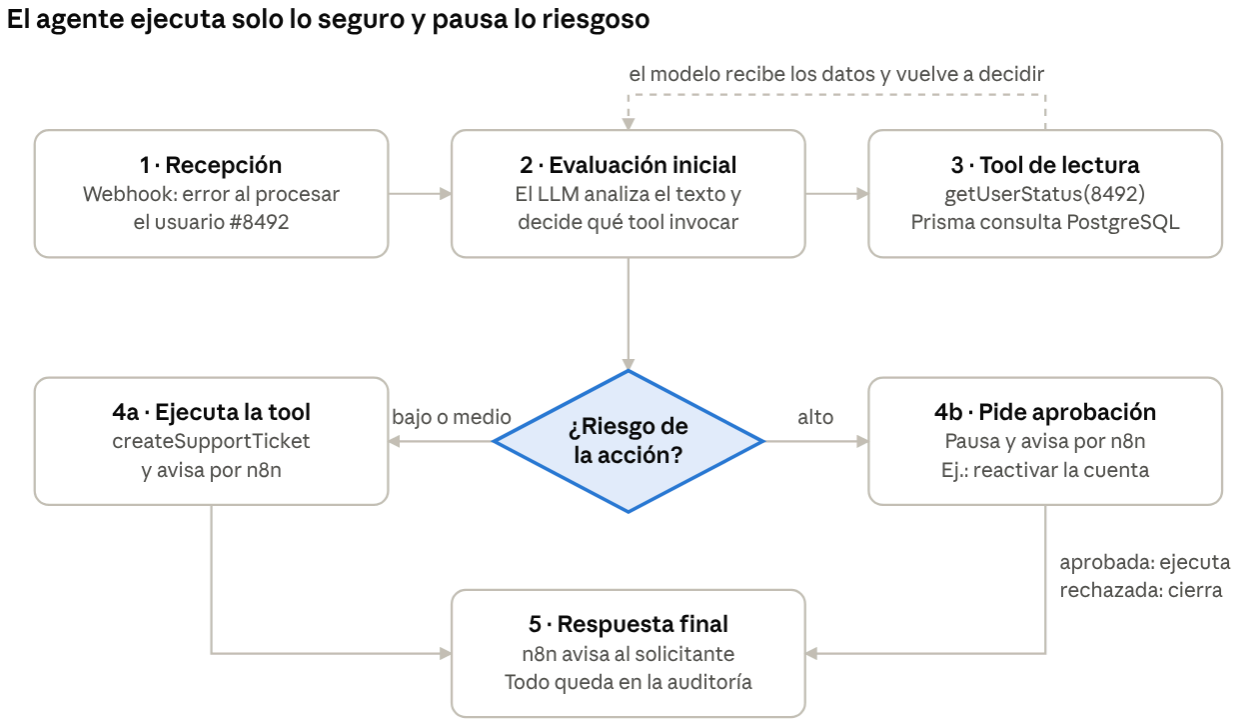
Arquitectura de componentes



arquitectura · n8n, agente en TypeScript y servicios externos

Los eventos entran por n8n; el orquestador decide con el LLM y ejecuta tools validadas. Las acciones de riesgo alto vuelven a n8n para pedir aprobación, y los evals corren como un runner del agente en CI. El LLM se usa siempre a través de una interfaz propia, así que cambiar de proveedor no toca el resto del agente.

Ciclo agéntico



ciclo agéntico · 5 pasos, 1 decisión, 1 loop

Un mismo requerimiento puede recorrer el loop varias veces: cada resultado de una tool vuelve al modelo hasta que decide actuar o responder.

En el ejemplo del usuario #8492, `getUserStatus` revela una cuenta suspendida. El agente crea el ticket con la categoría correcta, una acción de riesgo medio que se ejecuta sola, y n8n notifica el resultado. Reactivar la cuenta, en cambio, quedaría esperando aprobación humana.

Human-in-the-loop

El agente actúa solo en lecturas y acciones reversibles; cualquier modificación sensible queda en pausa hasta que una persona la apruebe.

Nivel de riesgo	Ejemplos de tools	Comportamiento
Bajo (lectura)	getUserStatus, consultarInventario	Se ejecuta automáticamente
Medio (escritura reversible)	createSupportTicket	Se ejecuta y notifica por n8n
Alto (modifica datos o cuentas)	reactivarCuenta, modificarUsuario	Pausa y exige aprobación humana

Cada tool declara su nivel de riesgo en el registro, así que el control no depende de que el modelo "decida" pedir permiso.

Flujo de aprobación:

1. El agente intenta ejecutar una tool de riesgo alto; el orquestador detecta el nivel y guarda la ejecución como `PENDING_APPROVAL` en PostgreSQL.
2. El agente llama a un webhook de n8n con el resumen de la acción propuesta.
3. n8n envía el mensaje a Slack o email con botones de aprobar y rechazar.
4. La respuesta vuelve al endpoint de callback del agente, que reanuda la ejecución o la cierra con el motivo del rechazo.
5. La decisión y quién la tomó quedan en la auditoría.

Seguridad y robustez

El modelo es una entrada no confiable: todo lo que genera y todo lo que lee se trata como dato a validar.

- **Validación con Zod:** los argumentos que el modelo propone para cada tool se validan antes de ejecutarla; si no cumplen el esquema, el error vuelve al modelo para que corrija.
- **Permisos mínimos:** cada tool accede solo a lo que necesita; el usuario de base de datos del agente tiene permisos acotados y las tools de lectura no pueden escribir.
- **Prompt injection:** el texto de la solicitud se delimita como dato en el prompt, nunca se interpreta como instrucciones, y las acciones de riesgo alto pasan siempre por aprobación humana.
- **Límites del loop:** máximo de iteraciones, timeout por ejecución y tope de reintentos para evitar bucles y costos descontrolados.
- **Idempotencia:** cada solicitud lleva un identificador único, de modo que un webhook reenviado no crea tickets duplicados.

- **Secretos:** claves de API y credenciales por variables de entorno, nunca en el repositorio ni en los logs.

Evaluación y métricas

Un dataset propio de casos de prueba permite medir el agente con números en lugar de afirmar que "funciona". Cada caso define la solicitud de entrada, las tools esperadas y el resultado correcto, y el runner de evals corre en GitHub Actions en cada cambio.

Métrica	Qué mide	Cómo se calcula
Resolución correcta	Casos donde el agente llega al resultado esperado	Casos correctos / total de casos
Selección de tools	Si invocó las herramientas adecuadas y con argumentos válidos	Coincidencia con las tools esperadas por caso
Escalamiento correcto	Si pidió aprobación en acciones de riesgo alto	Casos de riesgo alto que pasaron por aprobación / total de esos casos
Resistencia a inyección	Si ignoró instrucciones maliciosas dentro del texto de entrada	Casos adversarios resistidos / total de casos adversarios
Latencia	Tiempo hasta la respuesta final	Mediana y p95 por solicitud, tomadas de la auditoría
Costo	Gasto de tokens por solicitud	Tokens de entrada y salida × precio del modelo

El dataset debe incluir casos normales, casos ambiguos y casos adversarios. El mismo dataset se corre con Gemini y con Claude para comparar resolución, latencia y costo. Los resultados reales se publican en el README del repositorio.

Roadmap de implementación

Tres puertas de control separan las siete fases

1. **Setup y modelado de datos**
   Node + TypeScript, Prisma y PostgreSQL; tablas de usuarios, tickets, ejecuciones y auditoría
2. **Tools sin IA**
   Funciones con esquema Zod y nivel de riesgo, con tests en Vitest; consultan la BD o simulan acciones

◇ **Puerta: las tools pasan sus tests antes de conectar el modelo**

3. **Integración del motor LLM**
   Interfaz propia del LLM con adaptador de Gemini (Claude opcional), system prompt y esquema de tools
4. **Loop de control y resiliencia**
   El agente llama tools y decide solo; reintentos, límites de iteraciones y pausa en riesgo alto

◇ **Puerta: el agente resuelve de punta a punta el caso del usuario #8492**

5. **Workflows de n8n**
   Webhook de entrada, mensaje de aprobación con botones, callback al agente y notificaciones
6. **Evals y seguridad**
   Dataset de casos normales, ambiguos y adversarios; métricas de resolución, latencia y costo

◇ **Puerta: evals con resultados reales antes de empaquetar**

7. **Empaquetado y CI/CD**
   Dockerfile y docker-compose con agente, PostgreSQL y n8n; GitHub Actions; README y demo

roadmap · 7 fases, 3 puertas de control

Se mantienen las fases de la ficha original y se suman dos: los workflows de n8n (fase 5) y los evals con seguridad (fase 6). Cada puerta evita construir la siguiente capa sobre una base sin validar.

Entregables del portfolio y CV

Para el CV pesa más un repositorio que corre que esta ficha: estos son los entregables que lo respaldan.

- [ ] Repositorio en GitHub con README (arquitectura, cómo levantarlo con un comando, decisiones de diseño)
- [ ] Demo en video o GIF corto con el ciclo completo, incluida una aprobación humana desde n8n
- [ ] Workflows de n8n exportados en JSON dentro del repositorio
- [ ] Tests unitarios de las tools y resultados reales de los evals
- [ ] CI en verde con GitHub Actions

Cómo describirlo en el CV:

> Agente autónomo de triaje de soporte: backend en TypeScript/Node con tool calling (Gemini API, intercambiable por Claude), PostgreSQL + Prisma y auditoría completa del razonamiento. Human-in-the-loop para acciones sensibles mediante workflows de n8n. Dockerizado, con CI en GitHub Actions y evals sobre X casos (Y% de resolución correcta).

Reemplazar X e Y por los valores reales una vez corridos los evals.